

# 校园二手交易平台 — 数据库设计审查报告

审查范围：schema.sql (DDL)、data.sql (种子数据)、类图设计、后端实体类、API 规范文档  
审查维度：数据类型选择、索引设计、隐私数据处理  
审查日期：2026-05-15

## 一、数据库表结构总览

当前 schema.sql 共包含 10 张表：

| 序号 | 表名         | 用途   | 行数 (种子数据) |
|----|------------|------|-----------|
| 1  | user       | 平台用户 | 4         |
| 2  | admin      | 管理员  | 2         |
| 3  | good       | 二手商品 | 4         |
| 4  | order      | 订单主表 | 3         |
| 5  | order_item | 订单明细 | 3         |
| 6  | comment    | 商品评论 | 4         |
| 7  | rating     | 商品评分 | 3         |
| 8  | favorite   | 收藏夹  | 3         |
| 9  | report     | 举报记录 | 2         |
| 10 | good_image | 商品图片 | 4         |

## 二、数据类型选择审查

### 2.1 主键与 ID 字段

| 表  | 字段   | 当前类型                  | 评估                 | 建议                              |
|----|------|-----------------------|--------------------|---------------------------------|
| 全部 | *_id | INT<br>AUTO_INCREMENT | 对于校园级应用足够，约 21 亿上限 | 当前可接受。若预期用户量级达百万以上，建议升级为 BIGINT |

### 2.2 user / admin 表

| 字段           | 当前类型         | 评估                                               | 建议                                             |
|--------------|--------------|--------------------------------------------------|------------------------------------------------|
| username     | VARCHAR(100) | 实际用户名限制为 3-20 位 (InputSanitizer 校验)，100 略大但不浪费显著 | <input type="checkbox"/> 可接受                   |
| password     | VARCHAR(255) | BCrypt 哈希输出约 60 字符，255 完全够用                      | <input type="checkbox"/> 合理                    |
| email        | VARCHAR(120) | RFC 5321 规定邮箱最长 254 字符，120 可能截断长邮箱               | ⚠ 建议改为 VARCHAR(255)                            |
| phone        | VARCHAR(30)  | 支持国际号码格式                                         | <input type="checkbox"/> 合理                    |
| avatar_url   | VARCHAR(255) | 普通 URL 足够，但 CDN 签名 URL 可能超长                      | ⚠ 建议改为 VARCHAR(512)                            |
| campus       | VARCHAR(100) | 校名长度足够                                           | <input type="checkbox"/> 合理                    |
| address      | VARCHAR(255) | 地址通常够用                                           | <input type="checkbox"/> 合理                    |
| role (admin) | VARCHAR(50)  | 实际只有 SUPER_ADMIN 和 CONTENT_ADMIN 两种              | ⚠ 建议使用 ENUM('SUPER_ADMIN', 'CONTENT_ADMIN') 约束 |

### 2.3 good 表

| 字段          | 当前类型          | 评估                                         | 建议                                                                                        |
|-------------|---------------|--------------------------------------------|-------------------------------------------------------------------------------------------|
| title       | VARCHAR(150)  | 商品标题，150 足够                                | <input type="checkbox"/> 合理                                                               |
| description | TEXT          | 最大 64KB，适合长文本                              | <input type="checkbox"/> 合理                                                               |
| price       | DECIMAL(10,2) | 金融计算首选定点数，范围 ±99,999,999.99，校园交易完全够用       | <input type="checkbox"/> 正确选择                                                             |
| category    | VARCHAR(100)  | book/digital/daily/sports，用 VARCHAR 灵活但无约束 | <input checked="" type="checkbox"/> 建议<br>ENUM('book','digital','daily','sports','other') |
| status      | VARCHAR(50)   | 取值 ON_SALE/SOLD，无约束                        | <input checked="" type="checkbox"/> 建议<br>ENUM('ON_SALE','SOLD','OFF_SHELF')              |
| condition   | VARCHAR(50)   | 取值 USED_GOOD/USED_FAIR                     | <input checked="" type="checkbox"/> 建议<br>ENUM('USED_GOOD','USED_FAIR','USED_POOR')       |
| view_count  | INT DEFAULT 0 | 浏览次数计数器                                    | <input type="checkbox"/> 合理                                                               |

关于 ENUM vs VARCHAR 的选择：

- **ENUM 优点：**数据完整性约束、存储效率（1-2 字节）、自文档化
- **VARCHAR 优点：**灵活扩展，添加新类型不需 ALTER TABLE
- **建议：**当前阶段类目/状态取值已固定，使用 ENUM 可防止脏数据；未来如需动态扩展，可配合应用层枚举一起迁移

2.4 order / order\_item 表

| 字段           | 当前类型          | 评估            | 建议                                       |
|--------------|---------------|---------------|------------------------------------------|
| total_amount | DECIMAL(10,2) | 金额字段          | <input type="checkbox"/> 正确              |
| unit_price   | DECIMAL(10,2) | 单价            | <input type="checkbox"/> 正确              |
| subtotal     | DECIMAL(10,2) | 小计冗余存储，避免每次计算 | <input type="checkbox"/> 合理的反范式设计        |
| order_number | VARCHAR(100)  | UNIQUE 订单号    | <input type="checkbox"/> 合理，已有 UNIQUE 约束 |
| quantity     | INT DEFAULT 1 |               | <input type="checkbox"/> 合理              |

2.5 其他表

| 表          | 字段            | 当前类型          | 评估              | 建议                                                                     |
|------------|---------------|---------------|-----------------|------------------------------------------------------------------------|
| comment    | content       | VARCHAR(500)  | 中文评论 500 字符可能稍紧 | <input checked="" type="checkbox"/> 建议改为 TEXT                          |
| report     | reason        | VARCHAR(255)  | 举报理由可能较长        | <input checked="" type="checkbox"/> 建议改为 TEXT                          |
| report     | handle_result | VARCHAR(255)  | 处理结果也可能较长       | <input checked="" type="checkbox"/> 建议改为 TEXT                          |
| rating     | score         | INT           | 无范围约束           | <input checked="" type="checkbox"/> 建议添加 CHECK (score BETWEEN 1 AND 5) |
| good_image | image_url     | VARCHAR(255)  | 签名 URL 可能超过 255 | <input checked="" type="checkbox"/> 建议改为 VARCHAR(512)                  |
| good_image | sort_order    | INT DEFAULT 0 |                 | <input type="checkbox"/> 合理                                            |

2.6 时间字段：TIMESTAMP vs DATETIME

当前所有时间字段统一使用 `TIMESTAMP`：

| 特点 | TIMESTAMP               | DATETIME                |
|----|-------------------------|-------------------------|
| 范围 | 1970-01-01 ~ 2038-01-19 | 1000-01-01 ~ 9999-12-31 |
| 时区 | 自动转换 UTC                | 存储字面值                   |
| 存储 | 4 字节                    | 8 字节（MySQL 5.6+）        |

评估：对于校园项目，TIMESTAMP 足够且有时区自动转换优势。但需注意 **2038 年问题**。长期维护建议统一使用 DATETIME。

2.7 缺失表（类图有但 DDL 无）

| 缺失表                 | 类图参考                                        | 影响           | 建议                |
|---------------------|---------------------------------------------|--------------|-------------------|
| gold_transaction    | 无对应实体类，但 GoldService 引用了 GoldTransaction 概念 | 金币/积分流水无法持久化 | △ 如需实现信用分功能，应补充此表 |
| message             | Message 实体类                                 | 私信功能无存储支持    | 类图标注为"预留"，当前可延后   |
| verification_record | VerificationRecord 实体类                      | 实名认证无存储支持    | 类图标注为"预留"，当前可延后   |

三、索引设计审查

3.1 当前索引情况

当前 schema.sql 中显式定义的索引仅有一处：

| 表     | 索引           | 类型     | SQL                                       |
|-------|--------------|--------|-------------------------------------------|
| order | order_number | UNIQUE | order_number VARCHAR(100) NOT NULL UNIQUE |

MySQL InnoDB 引擎会自动为以下字段创建索引：

- 所有 PRIMARY KEY（聚簇索引）
- 所有 FOREIGN KEY 约束列（非聚簇索引）

当前 FK 自动索引覆盖的字段：

- good.seller\_id（FK → user）
- order.buyer\_id、order.seller\_id（FK → user）
- report.reporter\_id、report.target\_user\_id、report.target\_good\_id、report.handler\_admin\_id
- comment.user\_id、comment.good\_id、comment.parent\_comment\_id
- rating.user\_id、rating.good\_id
- favorite.user\_id、favorite.good\_id
- good\_image.good\_id
- order\_item.order\_id、order\_item.good\_id

3.2 缺失的关键索引

| 优先级 | 表       | 索引字段                  | 索引类型       | 原因                                                         |
|-----|---------|-----------------------|------------|------------------------------------------------------------|
| □ 高 | user    | username              | UNIQUE     | 登录查询的核心字段；注册时需唯一性校验。当前无此索引，每次登录都全表扫描                       |
| □ 高 | user    | email                 | UNIQUE     | 注册去重、邮箱登录、找回密码等功能依赖                                        |
| □ 中 | good    | (category, status)    | 复合索引       | 浏览商品页最频繁的查询：WHERE category = ? AND status = 'ON_SALE'      |
| □ 中 | good    | (status, created_at)  | 复合索引       | 首页最新商品列表：WHERE status = 'ON_SALE' ORDER BY created_at DESC |
| □ 中 | good    | seller_id             | （FK 已自动创建） | 查看"我发布的商品"，但 InnoDB FK 已覆盖 □                               |
| □ 中 | order   | (buyer_id, status)    | 复合索引       | 买家查"我的订单"并按状态筛选                                            |
| □ 中 | order   | (seller_id, status)   | 复合索引       | 卖家查"卖出的订单"并按状态筛选                                           |
| □ 中 | comment | (good_id, created_at) | 复合索引       | 查看某商品的所有评论，按时间排序                                           |
| □ 中 | report  | (status, created_at)  | 复合索引       | 管理员查看待处理举报列表                                               |

| 优先级                        | 表        | 索引字段               | 索引类型        | 原因              |
|----------------------------|----------|--------------------|-------------|-----------------|
| <input type="checkbox"/> 低 | favorite | (user_id, good_id) | UNIQUE 复合索引 | 防止同一用户重复收藏同一商品  |
| <input type="checkbox"/> 低 | rating   | (user_id, good_id) | UNIQUE 复合索引 | 防止同一用户对同一商品重复评分 |

### 3.3 索引设计汇总 SQL

```
-- === 高优先级 ===

-- 登录 / 注册核心索引
ALTER TABLE `user` ADD UNIQUE INDEX idx_user_username (username);
ALTER TABLE `user` ADD UNIQUE INDEX idx_user_email (email);

-- === 中优先级 ===

-- 商品浏览：按分类 + 状态筛选（最频繁查询）
ALTER TABLE good ADD INDEX idx_good_category_status (category, status);

-- 商品浏览：首页最新在售商品
ALTER TABLE good ADD INDEX idx_good_status_created (status, created_at);

-- 买家/卖家订单列表 + 状态筛选
ALTER TABLE `order` ADD INDEX idx_order_buyer_status (buyer_id, status);
ALTER TABLE `order` ADD INDEX idx_order_seller_status (seller_id, status);

-- 商品详情页评论列表
ALTER TABLE comment ADD INDEX idx_comment_good_created (good_id, created_at);

-- 管理员：待处理举报列表
ALTER TABLE report ADD INDEX idx_report_status_created (status, created_at);

-- === 低优先级 ===

-- 防重复收藏
ALTER TABLE favorite ADD UNIQUE INDEX idx_favorite_user_good (user_id, good_id);

-- 防重复评分
ALTER TABLE rating ADD UNIQUE INDEX idx_rating_user_good (user_id, good_id);
```

### 3.4 索引设计原则回顾

本设计遵循以下原则：

- 1. **最左前缀原则**：复合索引 (category, status) 可覆盖单独查 category 的查询，但需注意字段顺序
- 2. **高选择性优先**：username、email 选择性极高（几乎唯一），适合建索引；status 选择性较低（仅 3-4 种取值），通常放在复合索引的后面
- 3. **覆盖索引**：高频查询尽量让索引覆盖所需字段，避免回表
- 4. **避免过度索引**：每个索引都会增加写入成本（INSERT/UPDATE/DELETE 需维护索引），需权衡读写比例

## 四、隐私数据处理审查

### 4.1 密码存储

现状分析

| 层面                 | 现状                                                  | 评估                                                  |
|--------------------|-----------------------------------------------------|-----------------------------------------------------|
| DDL ( schema.sql ) | password VARCHAR(255)                               | <input type="checkbox"/> 字段长度能容纳 BCrypt 哈希 (~60 字符) |
| 种子数据 ( data.sql )  | 明文密码 '123456'                                       | <input type="checkbox"/> 严重问题                       |
| 后端 UserServiceImpl | 注册时 BCryptPasswordEncoder.encode()，登录时 matches() 验证 | <input type="checkbox"/> 正确实现                       |

| 层面                | 现状                               | 评估                                  |
|-------------------|----------------------------------|-------------------------------------|
| 后端 SecurityConfig | 注入 BCryptPasswordEncoder Bean    | <input type="checkbox"/> 正确配置       |
| User 实体           | @JsonIgnore 标注 password          | <input type="checkbox"/> 不会序列化到前端响应 |
| 防重复加密             | normalizeUser() 检测 \$2 前缀跳过已加密密码 | <input type="checkbox"/> 防御性设计      |

问题

1. 种子数据密码为明文：data.sql 中所有密码均为 '123456'，与 BCrypt 设计矛盾
- 影响：运行种子数据后，登录验证会失败（数据库存明文，代码期望 BCrypt 哈希）

◦ 修复：将种子数据中的密码替换为预生成的 BCrypt 哈希值
2. Admin 密码双重缺失：
- [Admin.java](#) 的 password 字段无 @JsonIgnore，存在密码泄露到前端响应的风险

◦ [AdminServiceImpl](#) 是空实现，未对管理员密码做 BCrypt 加密

修复建议

```
-- 预生成的 BCrypt 哈希（明文均为 "123456"，仅用于开发/测试）
-- 生产环境必须使用独立生成的随机哈希
UPDATE `user` SET password = '$2a$10$N9qo8uLOickgx2ZMRZoMyeIjZAgcf17p92ldGxad68LJZdL17lhWy';
UPDATE admin SET password = '$2a$10$N9qo8uLOickgx2ZMRZoMyeIjZAgcf17p92ldGxad68LJZdL17lhWy';

Admin 实体需添加 @JsonIgnore：
```

```
@JsonIgnore
private String password;
```

AdminServiceImpl 需在保存管理员时调用 passwordEncoder.encode()。

4.2 手机号保护

现状

| 表      | 字段    | 存储方式                          | 风险评估                     |
|--------|-------|-------------------------------|--------------------------|
| user   | phone | 明文 VARCHAR(30) <span>△</span> | 数据库中直接可读                 |
| API 响应 | phone | 无脱敏处理                         | <span>△</span> 前端展示完整手机号 |

是否需要加密？

| 考量因素  | 分析                                             |
|-------|------------------------------------------------|
| 数据敏感度 | 手机号属于《个人信息保护法》定义的个人信息，泄露可导致骚扰、诈骗               |
| 业务需要  | 买卖双方需要联系（面交），手机号是核心联系方式                        |
| 行业惯例  | 主流平台对手机号做脱敏展示（如 138****0001）+ 数据库加密存储（AES-256） |
| 本项目定位 | 校园二手交易，用户量有限，但需养成合规习惯                          |

建议方案

方案一（推荐，平衡安全与复杂度）：数据库明文 + API 脱敏展示

数据库：13800000001（明文）  
API 响应：138\*\*\*\*0001（脱敏）  
仅订单交易双方可查看完整手机号

方案二（高安全）：数据库 AES 加密 + API 脱敏

数据库：AES\_ENCRYPT('13800000001', secret\_key)  
API 响应：138\*\*\*\*0001  
后端解密仅在需要时执行

建议：当前阶段采用方案一（API 脱敏展示），后续升级到方案二。

API 脱敏实现示例：

```
// 手机号脱敏工具方法
public static function maskPhone(?string $phone): ?string
{
    if (empty($phone) || strlen($phone) < 7) {
        return $phone;
    }
    return substr($phone, 0, 3) . '****' . substr($phone, -4);
}
```

### 4.3 邮箱保护

| 考量   | 分析                                                 |
|------|----------------------------------------------------|
| 敏感度  | 低于手机号，但仍是个人信息                                      |
| 业务需要 | 注册/登录/通知，必须可读                                      |
| 建议   | 当前可明文存储。API 响应中可考虑部分脱敏展示（如 a****e@example.com），非必须 |

### 4.4 其他隐私考量

| 字段                            | 敏感度 | 建议               |
|-------------------------------|-----|------------------|
| address                       | 中   | 详细地址仅交易双方可见      |
| avatar_url                    | 低   | 可公开              |
| 设备指纹 device_fingerprint       | 中   | 仅风控后台可见，不对普通用户暴露 |
| IP 地址 ip_address (Withdrawal) | 中   | 仅风控/审计使用，不对前端返回  |

## 五、补充发现

### 5.1 数据一致性

| 问题                      | 详情                                       | 建议                |
|-------------------------|------------------------------------------|-------------------|
| data.sql 密码与 BCrypt 不一致 | 种子数据存明文，代码期望 BCrypt 哈希                   | 替换为预生成的 BCrypt 哈希 |
| order_item.subtotal 无约束 | subtotal = unit_price × quantity 依赖应用层保证 | 添加触发器或应用层校验       |

### 5.2 外键级联策略

当前所有 FK 均未指定 ON DELETE / ON UPDATE 行为，默认使用 InnoDB 的 NO ACTION（即 RESTRICT）：

| 场景              | 当前行为      | 建议                           |
|-----------------|-----------|------------------------------|
| 删除用户 → 其发布的商品   | 被 FK 阻止删除 | 合理，商品有独立生命周期                 |
| 删除商品 → 其图片      | 被 FK 阻止删除 | 建议 ON DELETE CASCADE，图片依附于商品 |
| 删除商品 → 收藏/评论/评分 | 被 FK 阻止删除 | 建议 ON DELETE CASCADE 或软删除    |
| 删除订单 → 订单明细     | 被 FK 阻止删除 | 建议 ON DELETE CASCADE         |

建议修改：

```
-- 商品图片：商品删除时级联删除
ALTER TABLE good_image DROP FOREIGN KEY fk_good_image_good;
ALTER TABLE good_image ADD CONSTRAINT fk_good_image_good
    FOREIGN KEY (good_id) REFERENCES good (good_id) ON DELETE CASCADE;
```

```
-- 订单明细：订单删除时级联删除
ALTER TABLE order_item DROP FOREIGN KEY fk_order_item_order;
ALTER TABLE order_item ADD CONSTRAINT fk_order_item_order
    FOREIGN KEY (order_id) REFERENCES `order` (order_id) ON DELETE CASCADE;
```

5.3 字符集

| 配置          | 当前值                | 评估                                            |
|-------------|--------------------|-----------------------------------------------|
| 数据库字符集      | utf8mb4            | <input type="checkbox"/> 支持 emoji 和全部 Unicode |
| 排序规则        | utf8mb4_unicode_ci | <input type="checkbox"/> 大小写不敏感，适合通用场景        |
| 表 SET NAMES | utf8mb4            | <input type="checkbox"/>                      |

六、审查结论与行动清单

6.1 必须修复（阻塞上线）

| 序号 | 问题                              | 影响          | 修复位置                       |
|----|---------------------------------|-------------|----------------------------|
| 1  | data.sql 密码为明文，与 BCrypt 验证不兼容   | 种子数据运行后无法登录 | data.sql                   |
| 2  | user.username 缺少 UNIQUE 索引      | 登录查询全表扫描    | schema.sql                 |
| 3  | Admin 实体 password 无 @JsonIgnore | 管理员密码泄露到前端  | <a href="#">Admin.java</a> |
| 4  | AdminServiceImpl 未加密密码          | 管理员密码明文存储   | AdminServiceImpl.java      |

6.2 强烈建议（影响性能/安全）

| 序号 | 问题               | 建议                                           |
|----|------------------|----------------------------------------------|
| 5  | 缺少 good 复合索引     | 添加 (category, status) 和 (status, created_at) |
| 6  | 缺少 order 复合索引    | 添加 (buyer_id, status) 和 (seller_id, status)  |
| 7  | 缺少 favorite 唯一索引 | 添加 UNIQUE (user_id, good_id) 防重复收藏           |
| 8  | 缺少 rating 唯一索引   | 添加 UNIQUE (user_id, good_id) 防重复评分           |
| 9  | 手机号 API 无脱敏      | 实现 maskPhone() 工具方法                          |

6.3 优化建议（锦上添花）

| 序号 | 问题                       | 建议                               |
|----|--------------------------|----------------------------------|
| 10 | email VARCHAR(120) 可能截断  | 改为 VARCHAR(255)                  |
| 11 | category/status/role 无约束 | 考虑使用 ENUM                        |
| 12 | FK 缺少级联策略                | 组合关系表添加 ON DELETE CASCADE        |
| 13 | score 无 CHECK 约束         | 添加 CHECK (score BETWEEN 1 AND 5) |
| 14 | rating 无 score CHECK     | 添加约束                             |
| 15 | gold_transaction 表缺失     | 如需积分功能，补充 DDL                    |

七、审查后 schema.sql（建议修订版）

以下为整合以上建议后的修订版 DDL（高亮 [NEW] / [CHANGED] 标记的变更）：

```
CREATE DATABASE IF NOT EXISTS nju_market DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE nju_market;

-- [CHANGED] email 长度从 120 改为 255
CREATE TABLE IF NOT EXISTS `user` (
  user_id INT PRIMARY KEY AUTO_INCREMENT,
  username VARCHAR(100) NOT NULL,
  password VARCHAR(255) NOT NULL,
  email VARCHAR(255),
  phone VARCHAR(30),
  avatar_url VARCHAR(512),
  campus VARCHAR(100),
  address VARCHAR(255),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  UNIQUE INDEX idx_user_username (username),
  -- [CHANGED] 支持长签名URL
  -- [NEW] 登录核心索引
```

-- [NEW] 注册去重索引

```
);
UNIQUE INDEX idx_user_email (email)

CREATE TABLE IF NOT EXISTS admin (
  admin_id INT PRIMARY KEY AUTO_INCREMENT,
  username VARCHAR(100) NOT NULL,
  password VARCHAR(255) NOT NULL,
  role ENUM('SUPER ADMIN', 'CONTENT ADMIN'), -- [CHANGED] 枚举约束
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS good (
  good_id INT PRIMARY KEY AUTO_INCREMENT,
  seller_id INT NOT NULL,
  title VARCHAR(150) NOT NULL,
  description TEXT,
  price DECIMAL(10, 2) NOT NULL,
  category ENUM('book', 'digital', 'daily', 'sports', 'other'), -- [CHANGED]
  status ENUM('ON SALE', 'SOLD', 'OFF SHELF'), -- [CHANGED]
  `condition` ENUM('USED_GOOD', 'USED_FAIR', 'USED_POOR'), -- [CHANGED]
  view_count INT DEFAULT 0,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_good_seller FOREIGN KEY (seller_id) REFERENCES `user` (user_id),
  INDEX idx_good_category_status (category, status), -- [NEW]
  INDEX idx_good_status_created (status, created_at) -- [NEW]
);

CREATE TABLE IF NOT EXISTS `order` (
  order_id INT PRIMARY KEY AUTO_INCREMENT,
  buyer_id INT NOT NULL,
  seller_id INT NOT NULL,
  order_number VARCHAR(100) NOT NULL UNIQUE,
  total_amount DECIMAL(10, 2) NOT NULL,
  status ENUM('PENDING', 'COMPLETED', 'CANCELLED'), -- [CHANGED]
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_order_buyer FOREIGN KEY (buyer_id) REFERENCES `user` (user_id),
  CONSTRAINT fk_order_seller FOREIGN KEY (seller_id) REFERENCES `user` (user_id),
  INDEX idx_order_buyer_status (buyer_id, status), -- [NEW]
  INDEX idx_order_seller_status (seller_id, status) -- [NEW]
);

CREATE TABLE IF NOT EXISTS report (
  report_id INT PRIMARY KEY AUTO_INCREMENT,
  reporter_id INT NOT NULL,
  target_user_id INT,
  target_good_id INT,
  report_type ENUM('USER', 'GOOD'), -- [CHANGED]
  reason TEXT, -- [CHANGED] 支持长文本
  status ENUM('PROCESSING', 'RESOLVED', 'REJECTED'), -- [CHANGED]
  handler_admin_id INT,
  handle_result TEXT, -- [CHANGED]
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_report_reporter FOREIGN KEY (reporter_id) REFERENCES `user` (user_id),
  CONSTRAINT fk_report_target_user FOREIGN KEY (target_user_id) REFERENCES `user` (user_id),
  CONSTRAINT fk_report_target_good FOREIGN KEY (target_good_id) REFERENCES good (good_id),
  CONSTRAINT fk_report_admin FOREIGN KEY (handler_admin_id) REFERENCES admin (admin_id),
  INDEX idx_report_status_created (status, created_at) -- [NEW]
);

CREATE TABLE IF NOT EXISTS comment (
  comment_id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT NOT NULL,
  good_id INT NOT NULL,
  parent_comment_id INT,
  content TEXT NOT NULL, -- [CHANGED] 支持长评论
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_comment_user FOREIGN KEY (user_id) REFERENCES `user` (user_id),
  CONSTRAINT fk_comment_good FOREIGN KEY (good_id) REFERENCES good (good_id),
  CONSTRAINT fk_comment_parent FOREIGN KEY (parent_comment_id) REFERENCES comment (comment_id),
  INDEX idx_comment_good_created (good_id, created_at) -- [NEW]
);

CREATE TABLE IF NOT EXISTS rating (
  rating_id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT NOT NULL,
  good_id INT NOT NULL,
  score INT NOT NULL CHECK (score BETWEEN 1 AND 5), -- [NEW] CHECK约束
```

```

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_rating_user FOREIGN KEY (user_id) REFERENCES `user` (user_id),
    CONSTRAINT fk_rating_good FOREIGN KEY (good_id) REFERENCES good (good_id),
    UNIQUE INDEX idx_rating_user_good (user_id, good_id)          -- [NEW] 防重复
);

CREATE TABLE IF NOT EXISTS favorite (
    favorite_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    good_id INT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_favorite_user FOREIGN KEY (user_id) REFERENCES `user` (user_id),
    CONSTRAINT fk_favorite_good FOREIGN KEY (good_id) REFERENCES good (good_id),
    UNIQUE INDEX idx_favorite_user_good (user_id, good_id)      -- [NEW] 防重复
);

CREATE TABLE IF NOT EXISTS good_image (
    image_id INT PRIMARY KEY AUTO_INCREMENT,
    good_id INT NOT NULL,
    image_url VARCHAR(512) NOT NULL,                          -- [CHANGED] 支持长URL
    sort_order INT DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_good_image_good FOREIGN KEY (good_id)
        REFERENCES good (good_id) ON DELETE CASCADE          -- [CHANGED] 级联删除
);

CREATE TABLE IF NOT EXISTS order_item (
    item_id INT PRIMARY KEY AUTO_INCREMENT,
    order_id INT NOT NULL,
    good_id INT NOT NULL,
    quantity INT NOT NULL DEFAULT 1,
    unit_price DECIMAL(10, 2) NOT NULL,
    subtotal DECIMAL(10, 2) NOT NULL,
    CONSTRAINT fk_order_item_order FOREIGN KEY (order_id)
        REFERENCES `order` (order_id) ON DELETE CASCADE,      -- [CHANGED] 级联删除
    CONSTRAINT fk_order_item_good FOREIGN KEY (good_id) REFERENCES good (good_id)
);

```

---

**审查结论：**数据库整体设计合理，DECIMAL 用于金额、utf8mb4 字符集等基础选择正确。主要需要在三个方面加强：索引设计（当前几乎空白）、密码存储一致性（种子数据与代码不匹配）、Admin 密码安全（缺少加密与序列化屏蔽）。建议按 6.1 节优先级完成修复后再进入下一阶段。